

THE COMPLETE GUIDE TO

Data Visualization

Plots, Anatomy, Statistics, AI/ML/DL & Python Code

Table of Contents

- 1. Bar Plots** — Anatomy, Types, Usage, Python Code
- 2. Box Plots** — Quartiles, Outliers, Whiskers, Python Code
- 3. Categorical Plots** — Strip, Swarm, Violin, Count, Point Plots
- 4. Regression Plots** — Linear, LOWESS, Polynomial, Residuals
- 5. Matrix Plots** — Heatmaps, Correlation Matrix, Confusion Matrix
- 6. Distribution Plots** — Histogram, KDE, ECDF, Q-Q Plot
- 7. Pie Charts** — Pie, Donut — When to Use & Avoid
- 8. Scatter Plots** — Bubble Charts, Pair Plots, t-SNE Embeddings
- 9. Line Plots** — Time Series, Training Curves, Area Charts
- 10. Additional Plots** — Violin, Hexbin, Area, ROC, Feature Importance
- 11. Axis, Titles & Styling** — Complete Professional Guide
- 12. Quick Reference** — Choose the Right Plot

1. Bar Plots

A bar plot (also called a bar chart) is one of the most fundamental and widely-used visualizations in data science. It uses rectangular bars — either horizontal or vertical — to represent the values of different categories. The length (or height) of each bar is proportional to the value it represents.

Think of it like a race: each category is a runner, and the height of the bar shows how fast (or how big) that runner is compared to others.

1.1 Anatomy of a Bar Plot

Part	Description
Title	Describes what the chart is about (e.g., 'Monthly Sales by Region')
X-Axis (horizontal)	Shows the categories (e.g., months, countries, product names). Label should name the category type.
Y-Axis (vertical)	Shows the numerical values. Should start at zero! Label should include units.
Bars	The rectangular shapes. Width is uniform; height encodes the value.
Bar Labels / Annotations	Optional numbers on top of bars for exact values.
Legend	Needed when bars are grouped or color-coded by a second variable.
Grid Lines	Horizontal lines behind bars help the eye trace values to the Y-axis.
Tick Marks	Small marks on axes indicating measurement intervals.
Color	Use distinct colors for different groups. Avoid rainbow colors — use a consistent palette.
Error Bars	Small vertical lines on top of bars showing uncertainty (standard deviation, confidence interval).

1.2 Types of Bar Plots

Type	Description	When to Use
Vertical Bar Chart	Bars go upward from the X-axis	Most common; use when category names are short
Horizontal Bar Chart	Bars go sideways from the Y-axis	When category names are long; easier to read labels
Grouped Bar Chart	Multiple bars side-by-side per category	Comparing sub-groups within each category
Stacked Bar Chart	Bars stacked on top of each other	Showing part-to-whole relationship across categories
100% Stacked Bar	Bars all reach 100%, showing proportions	Comparing proportions, not absolute values
Error Bar Chart	Bars with uncertainty indicators	Scientific data with measurement variance

1.3 Axis, Ticks, Title & Legend Details

X-Axis: Categorical — each tick corresponds to one category. Rotate tick labels 45° or 90° when names are long. Avoid crowding.

Y-Axis: Always start at 0 (truncated Y-axes are misleading!). Use clean intervals (0, 10, 20... not 0, 7, 14...). Add unit labels.

Title: Be specific — not 'Bar Chart' but 'Average Monthly Rainfall by City (2023)'.

Legend: Only needed for grouped/stacked/color-coded bars. Place outside the plot area. Keep it concise.

1.4 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Comparing group means or counts	Test score averages by school
Probability	Visualizing probability distributions over discrete events	Probability of each outcome when rolling a die
Data Science	EDA — exploring category frequencies	Number of records per category in a dataset
Machine Learning	Feature importance visualization	Which features matter most in a Random Forest
Deep Learning	Comparing model performance metrics	Accuracy of CNN vs RNN vs Transformer
Business Analytics	Sales by product, region, or time period	Q1 revenue per product line

1.5 When to Use / When NOT to Use

USE Bar Plots When...

- You have categorical data (non-numeric groups)
- You want to compare values across a small number of categories (ideally under 15)
- Your data has discrete, distinct groups (not continuous)
- You want to show counts, sums, averages, or frequencies

Do NOT Use Bar Plots When...

- You have continuous data — use a histogram instead
- You have too many categories (50+ bars become unreadable)
- You want to show change over time with continuous time — use a line plot
- You want to show relationships between two numerical variables — use scatter plots

1.6 Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# --- Basic Vertical Bar Chart ---
categories = ['Apples', 'Bananas', 'Cherries', 'Dates', 'Elderberries']
values = [23, 45, 12, 67, 34]

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(categories, values, color=sns.color_palette('Blues_d', len(categories)),
              edgecolor='white', linewidth=1.2)

# Add value labels on top of each bar
for bar in bars:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            str(int(bar.get_height())), ha='center', va='bottom', fontsize=10)

ax.set_title('Fruit Sales Comparison', fontsize=14, fontweight='bold', pad=15)
ax.set_xlabel('Fruit Type', fontsize=12)
ax.set_ylabel('Units Sold', fontsize=12)
ax.set_ylim(0, max(values) * 1.15) # Give room for labels
ax.yaxis.grid(True, linestyle='--', alpha=0.7)
ax.set_axisbelow(True) # Grid behind bars
plt.tight_layout()
plt.show()

# --- Grouped Bar Chart ---
df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr'],
    'Product_A': [150, 200, 175, 220],
    'Product_B': [100, 130, 160, 140]
})
x = np.arange(len(df['Month']))
width = 0.35
fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(x - width/2, df['Product_A'], width, label='Product A', color='#2D6A9F')
ax.bar(x + width/2, df['Product_B'], width, label='Product B', color='#E07B2A')
ax.set_xticks(x); ax.set_xticklabels(df['Month'])
ax.set_title('Monthly Sales by Product', fontsize=14, fontweight='bold')
ax.legend(); plt.tight_layout(); plt.show()

# --- Horizontal Bar Chart (for long labels) ---
countries = ['United States', 'China', 'Germany', 'United Kingdom', 'India']
gdp = [21.43, 14.34, 3.84, 2.83, 2.87]
fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(countries, gdp, color='#1B3A6B', edgecolor='white')
ax.set_xlabel('GDP (Trillion USD)', fontsize=12)
ax.set_title('Top 5 Economies by GDP', fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()
```

2. Box Plots

A box plot (also called a box-and-whisker plot) is a powerful visualization for showing the distribution and spread of numerical data. Unlike bar charts that show averages, box plots show you the entire statistical summary of your data in one compact picture: the minimum, maximum, median, and quartiles.

Think of it as a 5-number summary drawn as a picture. It tells you where most of your data sits, how spread out it is, and whether there are any unusual outlier values.

2.1 Anatomy of a Box Plot

Part	Description & Meaning
Box (Rectangle)	The central box spans from Q1 (25th percentile) to Q3 (75th percentile). This range is called the Interquartile Range (IQR). 50% of all data lies inside this box.
Median Line	The horizontal line inside the box — this is the median (Q2, 50th percentile). If it's near the center, data is symmetric. If skewed, the line shifts toward one end.
Whiskers	Lines extending from the box edges. Usually extend to $Q1 - 1.5 \times IQR$ (lower) and $Q3 + 1.5 \times IQR$ (upper). Whiskers show the range of non-outlier data.
Outliers (dots/circles)	Data points beyond the whiskers. Plotted as individual dots. These are unusually high or low values worth investigating.
IQR (Box width)	$IQR = Q3 - Q1$. A wide box means high spread in the middle 50% of data. Narrow = low spread.
Notches (optional)	V-shaped indentations in the box sides, showing confidence interval around the median. If two boxes' notches don't overlap, medians are significantly different.
Mean marker (optional)	A small triangle or X inside the box showing the mean. Compare with median to detect skewness.
X-Axis	Groups/categories being compared
Y-Axis	The numerical variable being measured
Title	Describe what distribution is shown and across what groups
Legend	Needed when multiple groups are shown with color coding

2.2 How to Read a Box Plot — Key Insights

What You See	What It Means	Action
Long box (tall IQR)	High variability in middle 50% of data	Investigate why data is so spread out
Median line near top of box	Data is left-skewed (more high values)	Check for skewness, may need log transform in ML
Many outlier dots	Data has extreme values	Decide whether to remove or investigate outliers

Two boxes at same level	Groups have similar medians	Groups may not be significantly different
Whisker much longer on one side	Distribution is skewed in that direction	Data not normally distributed
Tiny box (narrow IQR)	Data is tightly clustered in the middle	High consistency in that group

2.3 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Comparing distributions of groups	Blood pressure across age groups
Probability	Visualizing spread and identifying distribution shape	Is this data normally distributed?
Data Science (EDA)	Detecting outliers before modeling	Find extreme values in a numeric feature
Machine Learning	Comparing model prediction errors	Residuals distribution of different algorithms
Deep Learning	Comparing loss distributions across training runs	Epoch loss spread across 5 model runs
Feature Engineering	Understanding numeric feature distributions	Salary distribution by job title

2.4 When to Use / When NOT to Use

USE Box Plots When...

- You want to compare distributions across multiple groups
- You want to quickly identify skewness, spread, and outliers
- You have continuous numeric data
- You want a compact summary of large datasets

Do NOT Use Box Plots When...

- You have very small datasets (< 10 points) — individual data isn't hidden well
- You want to show the exact shape of distribution — use violin plot or histogram
- Your audience is unfamiliar with statistics — box plots can confuse non-technical viewers
- You have categorical/nominal data — no meaningful numerical spread to show

2.5 Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# --- Basic Box Plot ---
np.random.seed(42)
data = {
    'Group A': np.random.normal(70, 10, 100),
    'Group B': np.random.normal(80, 20, 100),
    'Group C': np.random.normal(60, 5, 100)
}

fig, ax = plt.subplots(figsize=(8, 6))
bp = ax.boxplot(data.values(), labels=data.keys(),
                patch_artist=True,          # Fill boxes with color
                notch=True,                 # Add median confidence interval notch
                medianprops={'color': 'red', 'linewidth': 2},
                flierprops={'marker': 'o', 'color': 'gray', 'alpha': 0.5})

colors = ['#EBF5FB', '#FEF9E7', '#EAF1F1']
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)

ax.set_title('Score Distribution by Group', fontsize=14, fontweight='bold')
ax.set_xlabel('Group', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.yaxis.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout(); plt.show()

# --- Seaborn Box Plot (easier syntax) ---
import pandas as pd
df = sns.load_dataset('tips') # Built-in dataset

fig, ax = plt.subplots(figsize=(9, 6))
sns.boxplot(data=df, x='day', y='total_bill', hue='sex',
            palette='Set2', linewidth=1.5, ax=ax)
ax.set_title('Restaurant Bill Distribution by Day and Gender',
            fontsize=13, fontweight='bold')
ax.set_xlabel('Day of Week', fontsize=11)
ax.set_ylabel('Total Bill ($)', fontsize=11)
ax.legend(title='Gender', loc='upper left')
plt.tight_layout(); plt.show()
```

3. Categorical Plots

Categorical plots are a family of visualizations designed specifically for data that falls into discrete groups or categories. Unlike numerical data that flows on a continuous scale, categorical data has distinct labels like 'Male/Female', 'Red/Blue/Green', or 'Low/Medium/High'. Seaborn's categorical module offers a rich set of plot types for this.

3.1 Types of Categorical Plots

Plot Type	Description
Strip Plot	Shows every individual data point as a dot per category. Like a scatter plot but one axis is categorical. Reveals the actual distribution.
Swarm Plot	Like a strip plot but points are adjusted so they don't overlap. Shows density and spread clearly without overlapping dots.
Box Plot	5-number summary (see Section 2). Great for comparing spread and outliers.
Violin Plot	A box plot combined with a kernel density estimate. Shows the full distribution shape on both sides, like a mirrored mountain.
Boxen Plot	An enhanced box plot with multiple quartile boxes. Great for larger datasets where box plots may miss details.
Point Plot	Shows the mean (or another estimator) with confidence intervals as dots connected by lines. Emphasizes the estimate.
Bar Plot (Categorical)	Shows the mean of a numeric variable per category with confidence intervals (different from a count bar chart).
Count Plot	Counts the number of observations in each category. The Y-axis shows frequency/count.

3.2 Anatomy of Categorical Plots

Part	Meaning in Categorical Plots
X-Axis	Categorical axis — each position is a label, not a number. Tick labels show category names. Rotate if long.
Y-Axis	Numeric axis — the measured variable. Always label with unit. In count plots, Y is frequency.
Title	State what categories are being compared and what is measured
Hue (color grouping)	A third variable encoded as color — allows comparing two categorical dimensions at once
Jitter	Small random noise added to X positions in strip plots so overlapping points become visible
Confidence Intervals	In bar/point plots, error bars or shaded bands showing uncertainty around the mean
Legend	Explains the hue/color variable. Should not repeat axis information.

Order parameter	The sequence in which categories are displayed — always think about logical ordering
-----------------	--

3.3 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Visualizing group differences in ANOVA tests	Test scores by teaching method
Data Science EDA	Understanding label distributions before training	Class imbalance in classification datasets
Machine Learning	Comparing model performance per class	Precision/Recall per category
Business Analytics	Customer segmentation visualization	Revenue by customer tier
Survey Analysis	Visualizing Likert scale responses	Agreement levels per question
NLP/Text Analytics	Comparing sentiment across categories	Positive/Negative tweets by brand

3.4 When to Use / When NOT to Use

USE Categorical Plots When...

- One axis is categorical (labels/groups) and the other is numeric
- You want to compare distributions, means, or counts across groups
- You have nominal or ordinal categorical variables
- You want to detect class imbalance in ML datasets (count plot)

Do NOT Use Categorical Plots When...

- Both axes are numeric — use scatter plot or line plot
- You have too many categories (20+) — consider grouping or using a table
- Your categories have natural order/time — use line plots

3.5 Python Code

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('tips')

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Categorical Plot Gallery', fontsize=16, fontweight='bold')
```

```
# Strip Plot
sns.stripplot(data=df, x='day', y='total_bill', ax=axes[0,0],
              jitter=True, palette='Set2', alpha=0.6)
axes[0,0].set_title('Strip Plot')

# Swarm Plot
sns.swarmplot(data=df, x='day', y='total_bill', ax=axes[0,1],
              palette='Set1', size=4)
axes[0,1].set_title('Swarm Plot')

# Violin Plot
sns.violinplot(data=df, x='day', y='total_bill', ax=axes[0,2],
               hue='sex', split=True, palette='pastel')
axes[0,2].set_title('Violin Plot')

# Box Plot
sns.boxplot(data=df, x='day', y='total_bill', ax=axes[1,0],
            palette='Blues')
axes[1,0].set_title('Box Plot')

# Point Plot
sns.pointplot(data=df, x='day', y='total_bill', ax=axes[1,1],
              hue='sex', palette='Set1', capsize=0.1)
axes[1,1].set_title('Point Plot')

# Count Plot
sns.countplot(data=df, x='day', ax=axes[1,2],
              palette='viridis', order=['Thur', 'Fri', 'Sat', 'Sun'])
axes[1,2].set_title('Count Plot')

plt.tight_layout(); plt.show()
```

4. Regression Plots

Regression plots show the relationship between two numerical variables along with a fitted regression line (or curve) that models that relationship. They combine a scatter plot with a model overlay, allowing you to simultaneously see the raw data AND the mathematical trend.

The regression line is the 'best fit' line that minimizes the total distance from all points to the line. It answers: 'If X goes up by 1, how much does Y change?'

4.1 Anatomy of a Regression Plot

Part	Description
Scatter Points	Individual data observations — each dot is one data point (X, Y pair)
Regression Line	The fitted line (linear: $y = mx + b$, or polynomial/LOWESS curve). Shows the central trend.
Confidence Band (shaded area)	The shaded region around the line showing uncertainty. Wider band = less confidence = more variance in data.
X-Axis (Predictor/Independent)	The input variable X. Should be labeled with variable name and units.
Y-Axis (Response/Dependent)	The output variable Y being predicted. Should be labeled with variable name and units.
Title	Should mention what is being regressed: 'Y vs X' or 'Effect of X on Y'
R-squared annotation	Sometimes added as text — shows how much variance in Y is explained by X (0 to 1)
Residuals	The vertical distance from each point to the regression line — smaller is better

4.2 Types of Regression Plots

Type	Description	When to Use
Simple Linear Regression	Straight line fit — $y = mx + b$	Linear relationship between two variables
Polynomial Regression	Curved line fit (quadratic, cubic...)	Non-linear but smooth relationship
LOWESS/LOESS	Locally weighted smoothing — flexible curve	Complex non-linear trends without assuming a shape
Logistic Regression Plot	S-shaped curve (sigmoid) for binary outcomes	Predicting probability of 0/1 outcome
Residual Plot	Shows errors (Y - predicted) vs fitted values	Checking if regression assumptions are met
Partial Regression Plot	Effect of one predictor after controlling others	Multiple regression — isolating one variable

4.3 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Hypothesis testing for linear association	Does temperature predict ice cream sales?
Probability	Visualizing conditional expectations	Expected income given years of education
Machine Learning	Visualizing model fit and predictions	Predicted vs actual house prices
Deep Learning	Visualizing learned regression curves	Neural network fit on Boston Housing dataset
Data Science EDA	Checking correlation before modeling	Does age correlate with churn rate?
Feature Selection	Seeing which features have linear relationships with target	Feature vs target regression plots

4.4 When to Use / When NOT to Use

USE Regression Plots When...

- Both variables are continuous/numeric
- You want to model or visualize a predictive relationship
- You're checking linearity assumptions before applying linear regression
- You want to communicate how strongly X predicts Y

Do NOT Use Regression Plots When...

- One or both variables are categorical — use bar or categorical plots
- You have no reason to believe there's a predictive relationship (e.g., unrelated variables)
- Data has multiple clusters — a single regression line will be meaningless
- You want to show correlation only (not prediction) — a simple scatter suffices

4.5 Python Code

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression

df = sns.load_dataset('tips')

# --- Seaborn regplot (simple) ---
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
sns.regplot(data=df, x='total_bill', y='tip',
            ax=axes[0],
            scatter_kws={'alpha': 0.5, 'color': '#2D6A9F'},
            line_kws={'color': '#C0392B', 'linewidth': 2},
            ci=95) # 95% confidence interval band
axes[0].set_title('Tip vs Total Bill (Linear Regression)', fontweight='bold')
axes[0].set_xlabel('Total Bill ($)'); axes[0].set_ylabel('Tip ($)')

# --- LOWESS (non-parametric smooth) ---
sns.regplot(data=df, x='total_bill', y='tip',
            ax=axes[1], lowess=True,
            scatter_kws={'alpha': 0.5, 'color': '#27AE60'},
            line_kws={'color': '#E07B2A', 'linewidth': 2})
axes[1].set_title('Tip vs Total Bill (LOWESS)', fontweight='bold')
axes[1].set_xlabel('Total Bill ($)'); axes[1].set_ylabel('Tip ($)')
plt.tight_layout(); plt.show()

# --- Polynomial Regression ---
x = np.linspace(0, 10, 100)
y = 2*x**2 - 3*x + 1 + np.random.normal(0, 5, 100)
fig, ax = plt.subplots(figsize=(8, 5))
sns.regplot(x=x, y=y, ax=ax, order=2, # order=2 for quadratic
            scatter_kws={'alpha': 0.4},
            line_kws={'color': 'red', 'linewidth': 2})
ax.set_title('Polynomial Regression (Degree 2)', fontweight='bold')
plt.tight_layout(); plt.show()
```

5. Matrix Plots

Matrix plots display data organized in a grid (matrix) format, where rows and columns represent different variables or categories, and the cell values (usually color-coded) show some relationship, correlation, or count between them. They are extremely powerful for understanding many variables at once.

5.1 Types of Matrix Plots

Type	Description
Heatmap	A color grid where each cell's color intensity encodes a numeric value. Used for correlations, confusion matrices, frequency tables.
Correlation Matrix	A special heatmap showing pairwise correlation coefficients between all numeric variables (-1 to +1).
Confusion Matrix Heatmap	Used in ML classification — rows are actual classes, columns are predicted. Diagonal = correct predictions.
Cluster Map	A heatmap with hierarchical clustering applied to rows and columns — groups similar rows/columns together.
Pair Plot (Grid)	Not strictly a matrix, but shows pairwise scatter plots for all variable combinations in a grid.
Pivot Table Heatmap	Aggregated data (sum, mean, count) shown as a heatmap for two categorical variables.

5.2 Anatomy of a Heatmap

Part	Description
Color Scale / Colormap	The most critical element. Color encodes the cell value. Diverging colormaps (blue-white-red) for correlation. Sequential (light-dark) for counts/density.
Colorbar (Legend)	The vertical bar on the side showing what each color maps to numerically. Always include units.
Row Labels (Y-Axis)	Names of variables/categories on the vertical axis. Rotate if needed.
Column Labels (X-Axis)	Names of variables/categories on the horizontal axis.
Cell Annotations	Numbers printed inside each cell for exact values. Critical for confusion matrices.
Title	Describe what relationship is being shown: 'Correlation Matrix of Features'
Tick Marks	On both axes marking each row/column position. In heatmaps, every tick = one cell.

5.3 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Exploring multivariate correlations	Correlation between 10 economic indicators
Machine Learning	Evaluating classification model performance	Confusion matrix after model prediction
Feature Engineering	Identifying multicollinearity	Dropping highly correlated features before training
Deep Learning	Visualizing attention weights in Transformers	Attention heatmap in BERT
Data Science EDA	Finding patterns in aggregated data	Average sales by region and month
NLP	Co-occurrence matrix of words	Word-word frequency in a corpus

5.4 When to Use / When NOT to Use

USE Matrix Plots When...

- You want to show relationships between many variables simultaneously
- You have a correlation matrix to visualize
- You need to display a confusion matrix for classification results
- You want to show frequency/aggregation across two categorical dimensions

Do NOT Use Matrix Plots When...

- You have many variables (50+ columns become unreadable in a heatmap)
- The data is not structured as a grid
- You want to show individual data points — use scatter plots
- Exact values are more important than patterns — use a table instead

5.5 Python Code

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# --- Correlation Heatmap ---
df = sns.load_dataset('penguins').dropna()
numeric_df = df.select_dtypes(include='number')
corr_matrix = numeric_df.corr()

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr_matrix,
            annot=True,          # Show numbers in cells
```

```
        fmt='.2f',          # 2 decimal places
        cmap='RdBu_r',     # Red=negative, Blue=positive
        vmin=-1, vmax=1,  # Force full correlation range
        square=True,      # Keep cells square
        linewidths=0.5,   # Grid lines between cells
        cbar_kws={'label': 'Pearson Correlation'},
        ax=ax)
ax.set_title('Feature Correlation Matrix – Penguins Dataset',
            fontsize=13, fontweight='bold', pad=15)
plt.tight_layout(); plt.show()

# --- Confusion Matrix ---
from sklearn.metrics import confusion_matrix
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = RandomForestClassifier(); model.fit(X_train, y_train)
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Setosa', 'Versicolor', 'Virginica'],
            yticklabels=['Setosa', 'Versicolor', 'Virginica'], ax=ax)
ax.set_xlabel('Predicted Class'); ax.set_ylabel('Actual Class')
ax.set_title('Confusion Matrix – Iris Classification', fontweight='bold')
plt.tight_layout(); plt.show()
```

6. Distribution Plots

Distribution plots reveal the shape and spread of your data — showing where values are concentrated, how they spread out, whether they are symmetric or skewed, and if there are multiple peaks (modes). Understanding distributions is foundational to all of statistics and machine learning.

6.1 Types of Distribution Plots

Type	Description
Histogram	Divides the data range into equal-width bins and counts how many data points fall in each bin. Most fundamental distribution plot.
KDE Plot (Kernel Density Estimate)	A smooth curve version of a histogram. Uses mathematical kernels to estimate the probability density function of the data.
Dist Plot (histplot + KDE)	Combines histogram and KDE in one plot — shows both counts and smooth density.
ECDF (Empirical CDF)	Shows the cumulative proportion of data at or below each value. Goes from 0 to 1 on Y-axis.
Rug Plot	Tiny vertical lines at each data point along the axis. Shows exact positions of all observations.
Q-Q Plot	Quantile-Quantile plot — compares your data distribution to a theoretical one (usually normal). Points on diagonal = matches theory.
Violin Plot	Combines KDE with box plot statistics to show distribution shape per group.

6.2 Anatomy of a Distribution Plot (Histogram)

Part	Description
Bins (Bars)	Each bar = one interval (bin). Width = bin size. Height = count or density of values in that range.
Bin Width / Number of Bins	Critical parameter! Too few bins = oversimplified. Too many = noisy. Rule of thumb: $\sqrt{n}$ or Sturges' rule.
X-Axis	The variable being measured. Continuous scale. Label with variable name and unit.
Y-Axis	Either Count (frequency) or Density (probability). Label accordingly.
KDE Overlay	Optional smooth curve showing estimated probability density function.
Mean/Median Lines	Optional vertical lines marking central tendency.
Title	Name the variable and dataset: 'Distribution of Customer Ages'
Color / Alpha	Single color for one group; multiple transparent colors for comparing multiple distributions.

6.3 Reading Distribution Shapes

Shape	What It Looks Like	What It Means
Normal / Bell Curve	Symmetric, single peak in middle	Data clustered around mean; most ML algorithms assume this
Right-Skewed (Positive)	Long tail to the right, peak on left	Few very large values; common in income, prices
Left-Skewed (Negative)	Long tail to the left, peak on right	Few very small values
Bimodal	Two peaks	Two distinct subgroups in data (e.g., two age groups)
Uniform	Flat — all values equally likely	Random uniform data; rare in real world
Heavy-tailed	Normal but with fat tails	Extreme outliers more frequent than normal

6.4 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Checking normality assumptions before tests	Is this data normal enough for a t-test?
Probability	Estimating probability density functions	PDF of a continuous random variable
Data Science EDA	Understanding numeric feature shapes before modeling	Is 'age' right-skewed? Apply log transform.
Machine Learning	Checking prediction distributions	Are my predicted values reasonably distributed?
Deep Learning	Weight/gradient distribution monitoring	Are gradients vanishing (spike near zero)?
Feature Engineering	Deciding which transformation to apply	Log-transform right-skewed features for linear models

6.5 When to Use / When NOT to Use

USE Distribution Plots When...

- You want to understand the shape, center, and spread of continuous data
- You need to check normality assumptions before applying statistical tests
- You're deciding which transformation to apply to a feature
- You want to compare distributions between two or more groups

Do NOT Use Distribution Plots When...

- You have categorical data — use bar charts or count plots
- You want to compare specific values across groups — use box plots
- You have too few data points (< 30) — histograms become meaningless

6.6 Python Code

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Generate sample data
np.random.seed(42)
data_normal = np.random.normal(60, 15, 500)
data_skewed = np.random.exponential(scale=20, size=500)

fig, axes = plt.subplots(2, 2, figsize=(13, 9))
fig.suptitle('Distribution Plot Types', fontsize=15, fontweight='bold')

# Histogram with KDE
sns.histplot(data_normal, kde=True, ax=axes[0,0], color='#2D6A9F',
             bins=30, stat='density')
axes[0,0].axvline(np.mean(data_normal), color='red', linestyle='--', label='Mean')
axes[0,0].axvline(np.median(data_normal), color='green', linestyle='--',
                 label='Median')
axes[0,0].legend(); axes[0,0].set_title('Histogram + KDE (Normal Data)')

# KDE only - comparing two groups
sns.kdeplot(data_normal, ax=axes[0,1], label='Normal', fill=True, alpha=0.4)
sns.kdeplot(data_skewed, ax=axes[0,1], label='Skewed', fill=True, alpha=0.4)
axes[0,1].set_title('KDE Comparison: Normal vs Skewed')
axes[0,1].legend()

# ECDF
sns.ecdfplot(data_normal, ax=axes[1,0], label='Normal', linewidth=2)
sns.ecdfplot(data_skewed, ax=axes[1,0], label='Skewed', linewidth=2)
axes[1,0].set_title('ECDF Plot')
axes[1,0].set_xlabel('Value'); axes[1,0].set_ylabel('Cumulative Proportion')
axes[1,0].legend()

# Q-Q Plot (normality check)
import scipy.stats as stats
stats.probplot(data_normal, dist='norm', plot=axes[1,1])
axes[1,1].set_title('Q-Q Plot (Normality Check)')

plt.tight_layout(); plt.show()
```

7. Pie Charts

A pie chart is a circular chart divided into slices, where each slice represents a category's proportion of the whole. The angle (and area) of each slice is proportional to the value it represents — larger slices mean larger shares of the total.

Important Note: Pie charts are among the most misused charts in data visualization. Humans are poor at judging angles and areas, making it hard to compare slices of similar size. Use them sparingly and only in specific scenarios.

7.1 Anatomy of a Pie Chart

Part	Description
Slices (Wedges)	Each slice = one category. Angle is proportional to value. Together all slices = 100%.
Labels	Category names — can be placed inside slices, outside with lines (leaders), or in a legend.
Percentage Labels	Show the % each slice represents. Often more readable than raw values.
Title	State clearly what the whole represents: 'Market Share by Company (2024)'
Legend	Lists categories with matching colors. Use when slice labels would be cluttered.
Explode / Offset	A slice pulled out slightly from center to highlight it.
Start Angle	Where the first slice begins (often top, at 90°). Consistency matters.
Color	Each slice gets a distinct color. Avoid red-green combinations (colorblind unfriendly).
Donut Hole (Donut Chart)	A hole in the center converts a pie to a donut chart. Center can show total or key metric.

7.2 Pie vs Donut Charts

Feature	Pie Chart vs Donut Chart
Visual weight	Pie: Full circle — can feel heavy. Donut: Lighter appearance.
Center space	Donut has center for additional text (total, KPI value).
Readability	Donut slightly easier to read — less visual area to decode.
When to use Donut	When you want to display a summary metric in the center.

7.3 Use in Statistics, AI/ML/DS

Field	Use Case	Recommendation
Business Analytics	Market share distribution	OK for 3-5 major categories

Data Science EDA	Class distribution overview	Better to use bar/count plot for many classes
Survey Analysis	Showing survey response breakdown	Use if only 2-4 options
Reporting	Budget allocation visual	OK when exact % matter and categories are few
ML	Class imbalance visualization	Avoid — bar chart is clearer for comparison

7.4 When to Use / When NOT to Use

USE Pie Charts ONLY When...

- You have 2-5 categories maximum (more slices become unreadable)
- You want to emphasize part-to-whole relationships (not compare sizes)
- Your audience is non-technical and just needs a 'quick glance' picture
- The proportions are very different (e.g., 70%/20%/10%) — easy to distinguish

Do NOT Use Pie Charts When...

- You have more than 5-6 categories (slices become tiny and confusing)
- You need precise comparisons — humans are bad at judging angles
- Values are similar to each other — bars are much clearer
- You want to show change over time — use line plots
- You are doing ML/AI analysis — bar charts almost always outperform pie charts here

7.5 Python Code

```
import matplotlib.pyplot as plt
import numpy as np

# --- Basic Pie Chart ---
labels = ['Product A', 'Product B', 'Product C', 'Others']
sizes = [42, 28, 20, 10]
colors = ['#2D6A9F', '#E07B2A', '#27AE60', '#C0392B']
explode = (0.05, 0, 0, 0) # Slightly pull out 'Product A'

fig, axes = plt.subplots(1, 2, figsize=(13, 6))

wedges, texts, autotexts = axes[0].pie(
    sizes, explode=explode, labels=labels, colors=colors,
    autopct='%1.1f%%',          # Show percentage in each slice
    startangle=90,              # Start from top
    pctdistance=0.75,          # % label position
    wedgeprops={'edgecolor': 'white', 'linewidth': 2})
```

```
for autotext in autotexts:
    autotext.set_fontweight('bold')
    autotext.set_fontsize(10)

axes[0].set_title('Market Share by Product (Pie)', fontsize=13, fontweight='bold')

# --- Donut Chart ---
wedges2, texts2, autotexts2 = axes[1].pie(
    sizes, labels=labels, colors=colors,
    autopct='%1.1f%%', startangle=90,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2,
                'width': 0.6}) # width < 1 makes it a donut

# Add center text
axes[1].text(0, 0, 'Total\n100%', ha='center', va='center',
            fontsize=12, fontweight='bold', color='#1B3A6B')
axes[1].set_title('Market Share by Product (Donut)', fontsize=13, fontweight='bold')

plt.tight_layout(); plt.show()
```

8. Scatter Plots

A scatter plot places dots on a 2D plane where the X position encodes one numeric variable and the Y position encodes another. Each dot = one observation. The pattern of dots reveals the relationship (or lack thereof) between the two variables.

Scatter plots are the backbone of correlation analysis, regression, clustering visualization, and dimensionality reduction. They are arguably the most important plot in data science.

8.1 Anatomy of a Scatter Plot

Part	Description
Data Points (Dots)	Each dot = one row in your dataset. Position encodes two numeric values.
X-Axis	The first numeric variable (predictor, feature, or any variable). Must be labeled with name and unit.
Y-Axis	The second numeric variable (response, feature, or target). Must be labeled with name and unit.
Color (Hue)	A third variable encoded as color — shows group membership or a third numeric variable.
Size (Bubble)	A fourth variable encoded as dot size — creates a 'bubble chart'. Larger bubble = larger value.
Marker Style	Different shapes (circle, triangle, square) to distinguish groups without color.
Alpha (Transparency)	When many points overlap, transparency helps see density. Range: 0 (invisible) to 1 (solid).
Trend Line	An optional regression line or curve showing the overall direction of the relationship.
Annotations	Text labels on specific important points.
Title	State what two variables are being compared: 'Height vs Weight by Gender'
Legend	Explains color/size/shape encoding if used.
Grid Lines	Help the eye trace exact coordinates of points.

8.2 Types and Extensions of Scatter Plots

Type	Extra Encoding	When to Use
Simple Scatter	X vs Y positions only	Basic correlation check between two variables
Colored Scatter	Color = third categorical variable	Seeing clusters by group
Bubble Chart	Size = third numeric variable	3D relationships in 2D space
Marginal Scatter	Histograms/KDE on axes margins	See distributions of both variables alongside relationship

3D Scatter	Z-axis for third variable	When three numeric variables need spatial encoding
t-SNE / UMAP Plot	High-dimensional scatter in 2D	Visualizing ML embeddings and clusters
Pair Plot	Grid of scatter plots for all variable pairs	Multi-variable EDA — see all relationships at once

8.3 Interpreting Scatter Plot Patterns

Pattern	Shape	Meaning
Strong positive correlation	Dots rise from lower-left to upper-right	When X increases, Y tends to increase
Strong negative correlation	Dots fall from upper-left to lower-right	When X increases, Y tends to decrease
No correlation	Random cloud of dots	X and Y are not linearly related
Clusters	Groups of dots separated from each other	Multiple subpopulations in data
Outliers	Dots far from the main cloud	Unusual observations to investigate
Non-linear relationship	Curved pattern (U-shape, S-shape)	Linear correlation insufficient — try polynomial or transformation
Heteroscedasticity	Fan-shaped pattern (spread grows)	Variance of Y changes with X — violates regression assumptions

8.4 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Pearson/Spearman correlation visualization	Height vs weight correlation
ML Feature Analysis	Feature vs target visualization	Income vs credit score before regression model
Deep Learning	Visualizing embeddings (t-SNE/UMAP)	Word2Vec embeddings clustered by word type
Clustering ML	Visualizing cluster assignments	K-means clusters on customer spending data
Model Evaluation	Predicted vs actual scatter	Perfect model: dots on diagonal line
Anomaly Detection	Outlier visualization	Fraud detection: anomalous points far from cluster

8.5 When to Use / When NOT to Use

USE Scatter Plots When...

- Both variables are continuous/numeric
- You want to visualize correlation or relationship between two variables
- You want to identify clusters, patterns, or outliers
- You want to evaluate regression model predictions (predicted vs actual)

Do NOT Use Scatter Plots When...

- One variable is categorical — use categorical plots or bar charts
- You have millions of points — overplotting makes it unreadable (use hexbin or density plot)
- You want to show trends over time — use line plots

8.6 Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

df = sns.load_dataset('iris')

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- Colored Scatter Plot ---
colors = {'setosa': '#2D6A9F', 'versicolor': '#E07B2A', 'virginica': '#27AE60'}
for species, group in df.groupby('species'):
    axes[0].scatter(group['sepal_length'], group['sepal_width'],
                    label=species, color=colors[species],
                    alpha=0.7, edgecolors='white', linewidth=0.5, s=60)
axes[0].set_xlabel('Sepal Length (cm)', fontsize=11)
axes[0].set_ylabel('Sepal Width (cm)', fontsize=11)
axes[0].set_title('Iris Dataset - Sepal Measurements by Species',
                  fontweight='bold')
axes[0].legend(title='Species')
axes[0].grid(True, linestyle='--', alpha=0.5)

# --- Bubble Chart (size encodes third variable) ---
scatter = axes[1].scatter(df['sepal_length'], df['petal_length'],
                          c=df['petal_width'], # Color encodes petal width
                          s=df['petal_width'] * 80, # Size also encodes petal width
                          cmap='viridis', alpha=0.7, edgecolors='white', linewidth=0.5)
plt.colorbar(scatter, ax=axes[1], label='Petal Width (cm)')
axes[1].set_xlabel('Sepal Length (cm)', fontsize=11)
axes[1].set_ylabel('Petal Length (cm)', fontsize=11)
axes[1].set_title('Bubble Chart - Size/Color = Petal Width', fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout(); plt.show()
```

```
# --- Pair Plot (all variable combinations) ---
pair_plot = sns.pairplot(df, hue='species', palette='Set1',
                        plot_kws={'alpha': 0.6},
                        diag_kind='kde') # Diagonal shows KDE
pair_plot.figure.suptitle('Iris Pair Plot', y=1.02, fontsize=14)
plt.show()
```

9. Line Plots

A line plot (also called a line chart) connects data points with a continuous line, showing how a value changes across an ordered sequence — most commonly time. The line emphasizes continuity and trend, making it the go-to chart for time series, learning curves, and sequential data.

The key insight of a line plot is connectedness: the line implies that values between the plotted points are meaningful — that there's a continuous transition from one point to the next.

9.1 Anatomy of a Line Plot

Part	Description
Line(s)	The core element. Each line represents one series/variable over the X axis sequence.
Data Points (Markers)	Optional dots/squares at each actual data observation. Helps see where data was measured.
X-Axis	The ordered dimension — usually time (dates, epochs, steps) or another continuous variable.
Y-Axis	The measured variable. Label clearly with units. Start at 0 unless change over time is the focus.
Confidence Band / Shading	Shaded area around line showing variance, standard deviation, or confidence interval.
Multiple Lines	Multiple series on same plot — each line gets a distinct color. Max ~5-6 lines for readability.
Legend	Maps each line color to its label. Essential with multiple lines.
Vertical Reference Lines	Vertical dashed lines marking events (e.g., 'Model deployed', 'COVID began')
Annotations	Text boxes pointing to key events or turning points on the line
Title	Specify what is being tracked and over what period: 'Weekly Revenue — Q1 2024'
Grid Lines	Horizontal grid helps trace values to Y-axis. Vertical grid marks time intervals.
X-Axis Tick Format	For dates: use MM/DD/YYYY or month abbreviations. Rotate if needed.

9.2 Types of Line Plots

Type	Description
Simple Line Chart	One line, one variable over time. Baseline chart for trends.
Multi-Line Chart	Multiple series on one chart. Compare trends side-by-side.
Area Chart	Line plot with filled area below. Emphasizes magnitude over time.
Stacked Area Chart	Multiple filled areas stacked. Shows both individual and total over time.

Step Chart	Line connects points as horizontal-then-vertical steps. Used for discrete changes.
Smoothed Line	Moving average or LOWESS applied to reduce noise in noisy time series.
Training/Validation Curve	ML-specific — shows training and validation loss over epochs.

9.3 Use in Statistics, AI/ML/DS

Field	Use Case	Example
Statistics	Visualizing time series trends	Monthly unemployment rate over 10 years
Machine Learning	Training/validation loss curves	Loss vs epoch during model training
Deep Learning	Learning rate schedules	LR warm-up and cosine annealing curve
Data Science	KPI tracking over time	Daily active users, revenue trends
Signal Processing	Waveform visualization	EEG signal over time
Finance	Stock price charts	AAPL stock price 2020-2024
Evaluation	ROC curve (sorted thresholds)	True Positive Rate vs False Positive Rate

9.4 When to Use / When NOT to Use

USE Line Plots When...

- X-axis is time or an ordered sequential variable
- Data is continuous — connecting points implies meaningful intermediate values
- You want to show trends, rise/fall, or rate of change
- You're tracking ML training metrics (loss, accuracy) over epochs

Do NOT Use Line Plots When...

- X-axis is categorical and order doesn't matter — use bar charts
- Data points are not connected/ordered (no inherent sequence)
- You have too many lines (7+ lines become a 'spaghetti chart')
- You want to show distributions — use histograms or box plots

9.5 Python Code

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import numpy as np
import pandas as pd

# --- Basic Time Series Line Plot ---
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=52, freq='W')
revenue = 1000 + np.cumsum(np.random.randn(52) * 30)

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(dates, revenue, color='#2D6A9F', linewidth=2.5,
        marker='o', markersize=4, markerfacecolor='white',
        markeredgewidth=1, markeredgecolor='#2D6A9F', label='Weekly Revenue')

# Add shaded area below line
ax.fill_between(dates, revenue, alpha=0.15, color='#2D6A9F')

# Add event annotation
ax.axvline(pd.Timestamp('2024-06-01'), color='red', linestyle='--', alpha=0.7)
ax.text(pd.Timestamp('2024-06-03'), revenue.max() * 0.95,
        'New Campaign Launch', color='red', fontsize=9)

ax.set_title('Weekly Revenue - 2024', fontsize=14, fontweight='bold')
ax.set_xlabel('Date', fontsize=11)
ax.set_ylabel('Revenue ($)', fontsize=11)
ax.yaxis.grid(True, linestyle='--', alpha=0.6)
ax.tick_params(axis='x', rotation=45)
plt.tight_layout(); plt.show()

# --- ML Training Curve ---
epochs = range(1, 51)
train_loss = [1.5 * np.exp(-0.08 * e) + 0.1 + np.random.randn()*0.02 for e in epochs]
val_loss = [1.5 * np.exp(-0.06 * e) + 0.2 + np.random.randn()*0.04 for e in epochs]

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(epochs, train_loss, label='Training Loss', color='#2D6A9F', linewidth=2)
ax.plot(epochs, val_loss, label='Validation Loss', color='#E07B2A',
        linewidth=2, linestyle='--')
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Loss', fontsize=12)
ax.set_title('Model Training & Validation Loss', fontsize=13, fontweight='bold')
ax.legend(fontsize=11); ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout(); plt.show()
```

10. Additional Plot Types

Beyond the core plot families, data visualization includes many specialized charts. Each solves a specific problem that general-purpose charts cannot handle well.

10.1 Violin Plot

A violin plot is a hybrid between a KDE plot and a box plot. The outer shape shows the distribution (like a KDE), while inside it includes the median, IQR, and quartile information. The wider the violin at any point, the more data exists there.

Aspect	Details
Best For	Comparing full distributions across multiple groups
X-Axis	Categorical groups
Y-Axis	Numeric variable
Advantage over Box Plot	Shows the full distribution shape, not just summary statistics
Use in ML	Comparing prediction distributions of different models
Anatomy	Outer KDE shape + inner box + median line + whiskers

```
sns.violinplot(data=sns.load_dataset('tips'), x='day', y='total_bill',
               hue='sex', split=True, palette='muted', inner='quartile')
plt.title('Bill Distribution by Day and Sex'); plt.show()
```

10.2 Histogram (Detailed)

Already covered in Distribution Plots (Section 6), but worth noting: in ML, histograms of feature values are essential EDA. Bin count matters — use the Freedman-Diaconis rule for optimal bins: $2 * IQR * n^{(-1/3)}$.

10.3 Hexbin Plot

A hexbin (hexagonal bin) plot is a scatter plot designed for massive datasets. Instead of individual dots (which overlap and obscure), the 2D space is divided into hexagonal bins, and each bin is colored by the count (or density) of points inside it.

Aspect	Details
Best For	When scatter plot has overplotting (100,000+ points)
Color Encoding	Number or density of points in each hexagon
X-Axis	First numeric variable (continuous)
Y-Axis	Second numeric variable (continuous)
Advantage	Reveals density structure hidden by overplotting
Use in ML/DL	Visualizing huge prediction vs actual datasets

```
fig, ax = plt.subplots(figsize=(8,6))
x = np.random.randn(50000); y = x * 0.8 + np.random.randn(50000) * 0.5
hb = ax.hexbin(x, y, gridsize=40, cmap='YlOrRd', mincnt=1)
```

```
plt.colorbar(hb, label='Count')
ax.set_title('Hexbin Plot (50,000 points)')
plt.tight_layout(); plt.show()
```

10.4 Area Chart

An area chart is a line chart with the area below filled in. It emphasizes the magnitude of values over time, not just the trend. Stacked area charts show multiple series and how they combine to a total.

Aspect	Details
Best For	Time series where cumulative magnitude matters
Stacked version	Part-to-whole over time — like a pie chart animated over time
When NOT to use	When lines cross each other frequently (confusion with stacking)
Use in DS	Website traffic by source over time, Revenue by region

```
fig, ax = plt.subplots(figsize=(10,5))
x = range(12)
y1 = [30,35,32,40,45,50,48,55,60,58,65,70]
y2 = [20,22,25,24,28,30,32,35,33,38,40,42]
ax.stackplot(x, y1, y2, labels=['Product A','Product B'],
             colors=['#2D6A9F','#E07B2A'], alpha=0.8)
ax.set_xticks(x)
ax.set_xticklabels(['Jan','Feb','Mar','Apr','May','Jun',
                   'Jul','Aug','Sep','Oct','Nov','Dec'])
ax.legend(loc='upper left'); ax.set_title('Monthly Sales - Stacked Area')
plt.tight_layout(); plt.show()
```

10.5 Heatmap (Standalone)

Already covered in Matrix Plots (Section 5). A key additional use: in NLP and Deep Learning, heatmaps visualize attention weights in Transformer models — showing which tokens attend to which other tokens.

10.6 Pair Plot / Scatterplot Matrix

A pair plot creates a grid of scatter plots for every combination of numeric variables. The diagonal shows each variable's distribution (KDE or histogram). Essential for multi-variate EDA.

Aspect	Details
Best For	Initial EDA with multiple numeric features
Diagonal	KDE or histogram of each variable alone
Off-diagonal	Scatter plot of variable pairs
Hue	Color by a categorical variable to see clustering
Use in ML	Feature correlation exploration before model training
Limitation	Becomes slow with many variables (10+ columns)

```
import seaborn as sns
```

```
df = sns.load_dataset('penguins').dropna()
g = sns.pairplot(df, hue='species', palette='Set2',
                 diag_kind='kde', plot_kws={'alpha':0.6})
g.figure.suptitle('Penguin Dataset Pair Plot', y=1.02, fontsize=14)
plt.show()
```

10.7 Residual Plot

A residual plot shows the prediction errors (residuals = actual - predicted) on the Y-axis vs the fitted values on the X-axis. Used to diagnose if regression assumptions are satisfied. Ideally, residuals should be randomly scattered around zero with no pattern.

Pattern in Residuals	Problem Indicated
Funnel shape (spread increases)	Heteroscedasticity — variance not constant
Curved pattern (U-shape)	Non-linearity — model missing a quadratic term
Systematic deviation	Model is biased — missing an important feature
Random scatter around 0	Assumptions met — model is correctly specified

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.random.randn(200, 1)
y = 3 * X.ravel() + np.random.randn(200)
model = LinearRegression().fit(X, y)
y_pred = model.predict(X)
residuals = y - y_pred
plt.figure(figsize=(8,5))
plt.scatter(y_pred, residuals, alpha=0.5, color='#2D6A9F')
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel('Fitted Values'); plt.ylabel('Residuals')
plt.title('Residual Plot'); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

10.8 ROC Curve

The ROC (Receiver Operating Characteristic) curve is a ML-specific line chart that plots the True Positive Rate (Sensitivity) on the Y-axis against the False Positive Rate (1 - Specificity) on the X-axis across all classification thresholds. The Area Under the Curve (AUC) summarizes model performance: AUC = 1.0 is perfect, AUC = 0.5 is random.

```
from sklearn.metrics import roc_curve, auc
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=1000, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = RandomForestClassifier().fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
```

```
plt.figure(figsize=(7,6))
plt.plot(fpr, tpr, color='#2D6A9F', lw=2, label=f'ROC Curve (AUC = {roc_auc:.3f})')
plt.plot([0,1],[0,1], 'k--', lw=1, label='Random Classifier (AUC = 0.5)')
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=11)
plt.ylabel('True Positive Rate (Sensitivity)', fontsize=11)
plt.title('ROC Curve - Binary Classification', fontsize=13, fontweight='bold')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

10.9 Feature Importance Plot

A horizontal bar chart specific to ML that shows how much each feature contributes to a model's predictions. Used for model interpretability (XAI — Explainable AI). Often combined with SHAP values for detailed explanations.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
import pandas as pd

X, y = load_iris(return_X_y=True, as_frame=True)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

importances = pd.Series(model.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.barh(importances.index, importances.values,
               color='#2D6A9F', edgecolor='white')
ax.set_xlabel('Feature Importance (Gini)', fontsize=11)
ax.set_title('Random Forest Feature Importance - Iris', fontweight='bold')
for bar, val in zip(bars, importances.values):
    ax.text(val + 0.002, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=10)
plt.tight_layout(); plt.show()
```

11. Complete Guide: Titles, Axes, Ticks, Legends & Labels

Every element in a plot has a purpose. Poor labeling is the most common mistake in data visualization — even a beautiful chart is useless if the reader can't understand what it shows. This section provides complete guidance for every annotation element.

11.1 Titles

Guideline	Details
Be Specific	Not 'Bar Chart' but 'Average Monthly Rainfall by City (2023)'
Include the Variable	Mention what is being measured: 'Distribution of Customer Ages'
Include Context	Add dataset, date range, or group: 'Model Loss — LSTM vs GRU (50 epochs)'
Font Size	Title: 13-16pt. Larger than axis labels. Bold.
Position	Default: top center. Can be top-left for editorial style.
Subtitle	Add a subtitle (ax.set_title + fig.suptitle) for additional context

11.2 Axis Labels

Guideline	Details
Always Label Both Axes	No exceptions — unlabeled axes are uninformative
Include Units	NOT 'Age' but 'Age (years)'. NOT 'Revenue' but 'Revenue (USD Millions)'
Font Size	11-13pt. Slightly smaller than title.
X-Axis Special Cases	Time series: use date format. Categorical: rotate 45° if names are long.
Y-Axis	Almost always numeric. If it starts at 0, do NOT truncate without reason.
Dual Y-Axis	Use only when comparing two variables with different scales on one plot. Color-code axes to match lines.

11.3 Tick Marks and Tick Labels

Axis Type	Tick Spacing Rule	Label Format
Continuous Numeric	Choose clean intervals: 0, 25, 50, 75, 100 NOT 0, 23, 46, 69	Numbers with units if applicable
Time Series (Daily)	Weekly or monthly intervals depending on range	MMM DD or YYYY-MM-DD
Time Series (Monthly)	Quarterly or annual intervals for long ranges	MMM YYYY or Q1 2024
Categorical	One tick per category	Category name; rotate 45° if > 8 chars

Log Scale	Powers of 10: 1, 10, 100, 1000	Scientific notation or 10^n format
Percentage	0%, 25%, 50%, 75%, 100%	Include % sign on first and last label at minimum

11.4 Legends

Guideline	Details
When to Include	Only when multiple series/groups are encoded by color, style, or size
When NOT to Include	When there's only one series — label the line/bar directly instead
Position	Default: upper right. Move to avoid overlapping data. Consider outside plot.
Legend Title	Give the legend a meaningful title: not 'hue' but 'Species' or 'Model Type'
Order	Match the visual order in the plot (top-to-bottom for line endpoints)
Font Size	9-11pt. Smaller than axis labels. Don't let legend dominate the chart.
Avoid Duplicate Info	If axis already shows group labels, don't repeat them in legend

11.5 Color Guidance

Use Case	Recommended Colormap or Palette
Categorical (unordered)	Set1, Set2, tab10 — distinct colors for each group
Sequential (low to high)	Blues, Greens, YlOrRd, viridis — light=low, dark=high
Diverging (negative to positive)	RdBu, coolwarm, PiYG — used for correlation, difference
Colorblind Safety	Use viridis, cividis, or Set2 — avoid pure red-green combinations
Corporate/Presentation	Use your brand palette, 2-3 colors maximum
Too Many Colors = Bad	If you need > 8 colors, consider small multiples or faceting

11.6 Grid Lines

Type	When to Use
Horizontal grid only	Bar charts, line charts — helps trace values to Y-axis
Both horizontal + vertical	Scatter plots, heatmaps — helps trace both coordinates
No grid	Minimal/editorial style; geographic maps
Style	Use dashed (--) or dotted (:) lines at 30-50% opacity so they don't compete with data

11.7 Python: Complete Styling Template

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Set global style
plt.rcParams.update({
    'font.family': 'Arial',
    'font.size': 11,
    'axes.titlesize': 14,
    'axes.titleweight': 'bold',
    'axes.labelsize': 12,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'legend.fontsize': 10,
    'legend.title_fontsize': 11,
    'figure.dpi': 150,
    'axes.spines.top': False,    # Remove top border
    'axes.spines.right': False, # Remove right border
    'axes.grid': True,
    'grid.alpha': 0.4,
    'grid.linestyle': '--',
})

fig, ax = plt.subplots(figsize=(10, 6))

# Your plot here ...

# Titles & labels
ax.set_title('Main Title Here', fontsize=15, fontweight='bold', pad=15)
ax.set_xlabel('X Variable (units)', fontsize=12, labelpad=10)
ax.set_ylabel('Y Variable (units)', fontsize=12, labelpad=10)

# Tick formatting
ax.tick_params(axis='x', rotation=0, length=4)
ax.tick_params(axis='y', length=4)

# Legend
legend = ax.legend(title='Group Variable', loc='upper left',
                   framealpha=0.9, edgecolor='#BFCDD8')
legend.get_title().set_fontweight('bold')

# Grid behind data
ax.set_axisbelow(True)

# Add data source annotation
fig.text(0.01, -0.02, 'Source: Dataset Name | Analysis by: Your Name',
        ha='left', fontsize=8, color='gray')

plt.tight_layout()
plt.savefig('plot.png', dpi=300, bbox_inches='tight') # High-res save
plt.show()
```

12. Quick Reference: Choosing the Right Plot

The most important skill in data visualization is choosing the right chart for your data and question. Use this reference table to make that decision quickly.

Your Question / Data Type	Recommended Plot	Alternative
Compare values across categories	Bar Chart (vertical)	Lollipop chart, Dot plot
Compare distributions across groups	Box Plot or Violin Plot	Strip plot, Swarm plot
Show distribution of one numeric var	Histogram + KDE	ECDF, Violin plot
Show relationship between two numeric vars	Scatter Plot	Hexbin (large data), Regression plot
Show trend over time	Line Chart	Area chart, Smoothed line
Show part-to-whole (few categories)	Pie Chart / Donut	Stacked bar chart
Show part-to-whole over time	Stacked Area Chart	100% stacked bar
Visualize correlation matrix	Heatmap	Pair plot
Evaluate classification model	Confusion Matrix Heatmap	ROC Curve
Show feature importance	Horizontal Bar Chart	SHAP plot
Visualize high-dimensional data	t-SNE / UMAP Scatter	PCA biplot
Compare many numeric variable pairs	Pair Plot (sns.pairplot)	Correlation heatmap
Show count per category	Count Plot / Bar Chart	Treemap
Show model predictions vs actual	Scatter (predicted vs actual)	Residual plot
Massive scatter (overplotting)	Hexbin Plot	Contour/density plot
Show geographic data	Choropleth Map	Dot map, Bubble map
Show network/graph relationships	Network Graph	Chord diagram

Show ranked comparison (many items)

Horizontal Bar Chart (sorted)

Lollipop chart

Summary: The Golden Rules of Data Visualization

Always Remember These Principles

1. **TITLE:** Always add a specific, descriptive title. Never leave it blank.
2. **AXES:** Always label both axes with variable names AND units.
3. **START AT ZERO:** Bar charts must start at zero. Line charts may not need to.
4. **COLOR:** Use colorblind-safe palettes. Avoid rainbow colormaps for continuous data.
5. **LEGEND:** Only add a legend when multiple groups/series exist.
6. **GRID:** Use subtle grid lines to aid reading, not overwhelm the data.
7. **CLUTTER:** Remove chart junk — unnecessary borders, 3D effects, gradients.
8. **CONTEXT:** Add data source, date, and author info as footnote.
9. **RIGHT CHART:** Match the chart type to your data type and question.
10. **AUDIENCE:** Simplify for non-technical audiences; detail for technical ones.